Basic Types, Strings, Nullable Types, and Enumerations in C#

C# provides a rich type system for representing numeric values, text, optional values, and predefined sets of choices. Choosing an appropriate type is important because the type determines how data is stored, what operations are available, and how accurately values can be represented.

This section focuses on four related areas:

1. Numeric value types
2. Strings and text manipulation
3. Nullable value types
4. Enumerations and flag enumerations

---

1. Numeric Value Types

A **value type** stores its value directly. Numeric types such as **`int`**, **`long`**, **`float`**, **`double`**, and **`decimal`** are value types.

The choice of numeric type should depend on the kind of data being represented rather than simply choosing the largest available type.

1.1 Integral Types

Integral types represent whole numbers.

Common unsigned integral types include:

C# Type	.NET Type	Literal Suffix	Size
`byte`	**`System.Byte`**	—	8 bits
`ushort`	**`System.UInt16`**	—	16 bits
`uint`	**`System.UInt32`**	**`U`**	32 bits
`ulong`	**`System.UInt64`**	**`UL`**	64 bits

For general integer calculations, **`int`** and **`long`** are the commonly favored integral types. Other integer types are useful when interoperability or storage efficiency matters.

Literal suffixes

A suffix can tell the compiler how a numeric literal should be interpreted.

```
uint population = 4_000_000U;

ulong largeValue = 9_000_000_000UL;
```

Here, **U** identifies an unsigned integer literal and **UL** identifies an unsigned long literal.

Note — Numeric Separators

The _ characters improve readability and do not affect the numeric value. For example, **4_000_000** is equivalent to **4000000**.

---

2. Real Number Types

C# provides three important real-number types:

C# Type	.NET Type	Typical Suffix	Size	Typical Use
float	**System.Single**	**F**	32 bits	Graphics/scientific calculations
double	**System.Double**	**D**	64 bits	General floating-point calculations
decimal	**System.Decimal**	**M**	128 bits	Financial/base-10-sensitive calculations

float and **double** are floating-point types. **decimal** provides high-precision base-10-oriented arithmetic and is particularly appropriate for financial calculations.

Examples

```
float temperature = 23.5F;

double distance = 12345.6789;

decimal price = 49.99M;
```

The suffix is particularly important for **float** and **decimal**.

---

3. Floating-Point Rounding Errors

A critical property of **`float`** and **`double`** is that they internally represent numbers using **base 2**. Many decimal fractions cannot be represented exactly in binary.

For example:

```
float tenth = 0.1f;

float one = 1f;

Console.WriteLine(one - tenth * 10f);
```

It may seem that the result must be exactly:

```
0
```

However, the result can be approximately:

```
-1.490116E-08
```

The reason is that **`0.1`** cannot be represented exactly using this floating-point representation.

Practical Lab: Observe the Rounding Error

Create a console application and enter:

```
float tenth = 0.1f;

float one = 1f;

float result = one - tenth * 10f;

Console.WriteLine($"Result: {result}");
```

Run the program and examine the result.

Warning — Money and Floating Point

Do not assume that **`float`** or **`double`** can exactly represent every **`decimal`** fraction. For calculations where base-10 accuracy is important, such as monetary calculations, **`decimal`** is typically the more appropriate type.

---

4. Strings

A **`string`** represents a sequence of characters. Strings are used extensively for names, messages, addresses, file paths, user input, formatted output, and other textual information.

4.1 Creating Strings

The most common approach is a string literal:

```
string course = "C# Development";
```

Type inference can also be used:

```
var message = "Hello";
```

A string can be produced from another value:

```
string number = 42.ToString();
```

The ToString() call converts the integer value into its textual representation.

C# also provides constructors for specialized situations.

Repeating a Character

```
string separator = new string('*', 20);
Console.WriteLine(separator);
```

Output:

```
********************
```

The constructor receives:

```
Character to repeat
        ↓
new string('*', 20)
                ↑
          Number of times
```

Creating a String from Characters

```
char[] letters = { 'h', 'e', 'l', 'l', 'o' };
string word = new string(letters);
Console.WriteLine(word);
```

Output:

```
hello
```

The source introduces string construction from ordinary values, repeated characters, and character arrays.

---

5. Reading and Printing Strings

Console applications commonly interact with users through **`Console.ReadLine()`**, **`Console.Write()`**, and **`Console.WriteLine()`**.

5.1 Reading Input

```
Console.Write("Please enter your name: ");
string name = Console.ReadLine();
```

`Console.Write()` displays the prompt without automatically moving to a new line.

`Console.ReadLine()` waits for the user to enter text and press **Enter**.

5.2 Printing Output

```
Console.Write("Hello, {0}! ", name);
Console.WriteLine("Welcome to our party!");
```

`Write()` keeps subsequent output on the same line, whereas **`WriteLine()`** finishes by moving the cursor to the next line.

---

6. Escape Sequences and Unicode Characters

Some characters have special meanings inside C# string and character literals.

For example:

```
Console.WriteLine("Hello\nWorld");
```

`\n` represents a newline.

Other commonly encountered escape sequences include:

- **`\n`** **New line**
- **`\t`** **Horizontal tab**
- **`\\`** **Backslash**

- `\"` Double quote
- `\'` Single quote

Unicode characters can also be represented using hexadecimal character codes:

```
char copyrightSymbol = '\u00A9';

char omegaSymbol = '\u03A9';

char newLine = '\u000A';


Console.WriteLine(copyrightSymbol);

Console.WriteLine(omegaSymbol);
```

The **\u** escape allows a Unicode character to be specified using its hexadecimal code.

---

7. Verbatim Strings

Strings containing many backslashes can become difficult to read because each backslash normally needs escaping.

Consider a network path:

```
string path = "\\\\server\\fileshare\\helloworld.cs";
```

C# provides **verbatim strings**, prefixed with @:

```
string path = @"\\server\fileshare\helloworld.cs";
```

Both represent the same path.

Tip

Verbatim strings are particularly useful when working with text containing many backslashes because they reduce escape-character clutter.

---

8. String Interpolation

String interpolation provides a readable way to embed values or expressions inside a string.

An interpolated string begins with $, and expressions appear inside { }.

```
string name = "Alex";
```

```
int age = 20;

Console.WriteLine($"Name: {name}, Age: {age}");
```

Expressions can also be evaluated:

```
Console.WriteLine($"This is a test {4 + 1}");
```

Output:

```
This is a test 5
```

The expression inside `{ }` is evaluated before its result becomes part of the output.

---

9. Comparing Strings

String comparison can mean either:

- testing whether two strings are equal; or
- determining their relative ordering.

These are different operations.

9.1 Equality with ==

```
string str1 = "Hello";

string str2 = "Hello";

if (str1 == str2)

{

    Console.WriteLine("The strings are equal.");

}
```

The comparison is case-sensitive:

```
"Hello" == "hello"
```

evaluates to `false`.

The `Equals()` method can also perform equality checking:

```
if (str1.Equals(str2))
{
    Console.WriteLine("The strings are equal.");
}
```

The material treats this form of Equals() as having the same case-sensitive effect as ==.

9.2 Comparing Relative Order

`string.Compare()` can determine the relative ordering of two strings.

```
int result = string.Compare(str1, str2, true);
```

The third argument controls whether case is ignored in this overload.

The result follows this pattern:

```
result == 0   → strings compare as equal
result < 0    → str1 comes before str2
result > 0    → str1 comes after str2
```

Pitfall

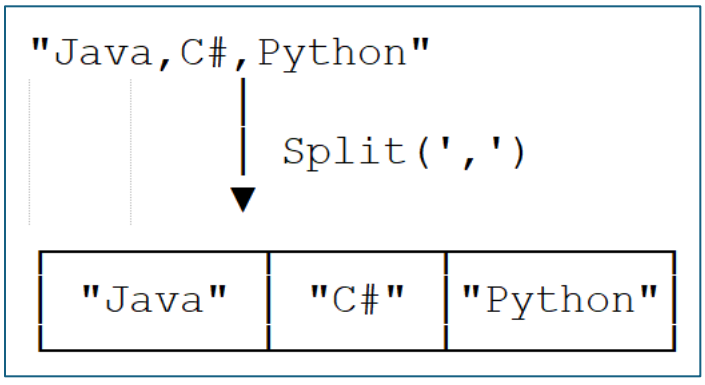
Do not assume the result will specifically be **-1**, **0**, or **1**. The important information is whether the returned integer is **negative, zero, or positive**.

---

10. Splitting Strings

`Split()` separates one string into multiple strings according to specified separator characters.

Conceptually:



Example

```
string languages = "Java,C#,Python";

string[] items = languages.Split(',');
```

```
foreach (string item in items)
{
    Console.WriteLine(item);
}
```

The source also demonstrates splitting on multiple characters:

```
string listOfBeers = "Amstel, Zagorka, Tuborg, Becks.";
string[] beers = listOfBeers.Split(' ', ',', '.');

foreach (string beer in beers)
{
    Console.WriteLine(beer);
}
```

The duplicate treatment of `Split()` in the source can be consolidated into this single section.

---

11. Joining Strings

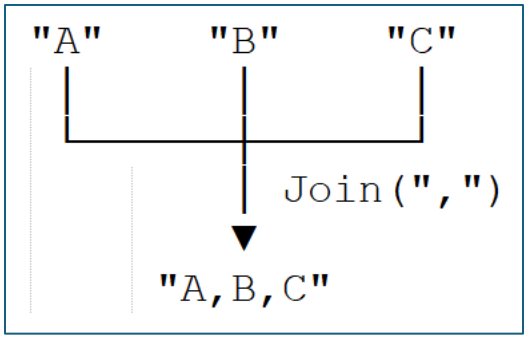
`String.Join()` performs the complementary operation: it combines multiple components while placing a separator between them.

```
var result = string.Join(",", "A", "B", "C");
Console.WriteLine(result);
```

Output:

```
A,B,C
```

Conceptually:



---

12. Changing Character Case

`ToLower()` converts characters to lowercase:

```
string alpha = "aBcDeFg";
string lowerAlpha = alpha.ToLower();

Console.WriteLine(lowerAlpha);
```

Output:

```
abcdefg
```

`ToUpper()` converts characters to uppercase:

```
string alpha = "aBcDeFg";
string upperAlpha = alpha.ToUpper();

Console.WriteLine(upperAlpha);
```

Output:

```
ABCDEFG
```

---

13. Trimming Strings

User input and imported text frequently contain unwanted whitespace.

13.1 `Trim()`

Removes whitespace from both ends:

```
string s = " example of white space ";
string clean = s.Trim();

Console.WriteLine(clean);
```

Result:

```
example of white space
```

13.2 Trimming Specific Characters

Specific characters can be supplied:

```
string s = " \t\nHello!!! \n";

string clean = s.Trim(' ', ',', '!', '\n', '\t');

Console.WriteLine(clean);
```

Output:

```
Hello
```

13.3 `TrimStart()` and `TrimEnd()`

Use these when characters should be removed from only one side:

```
string s = " C# ";
string clean = s.TrimStart();
```

The trailing space remains.

---

14. Practical Lab — Process Console Input

This short exercise combines several string operations.

Step 1 — Read a name

```
Console.Write("Enter your full name: ");
string name = Console.ReadLine();
```

Step 2 — Remove surrounding whitespace

```
name = name.Trim();
```

Step 3 — Produce different representations

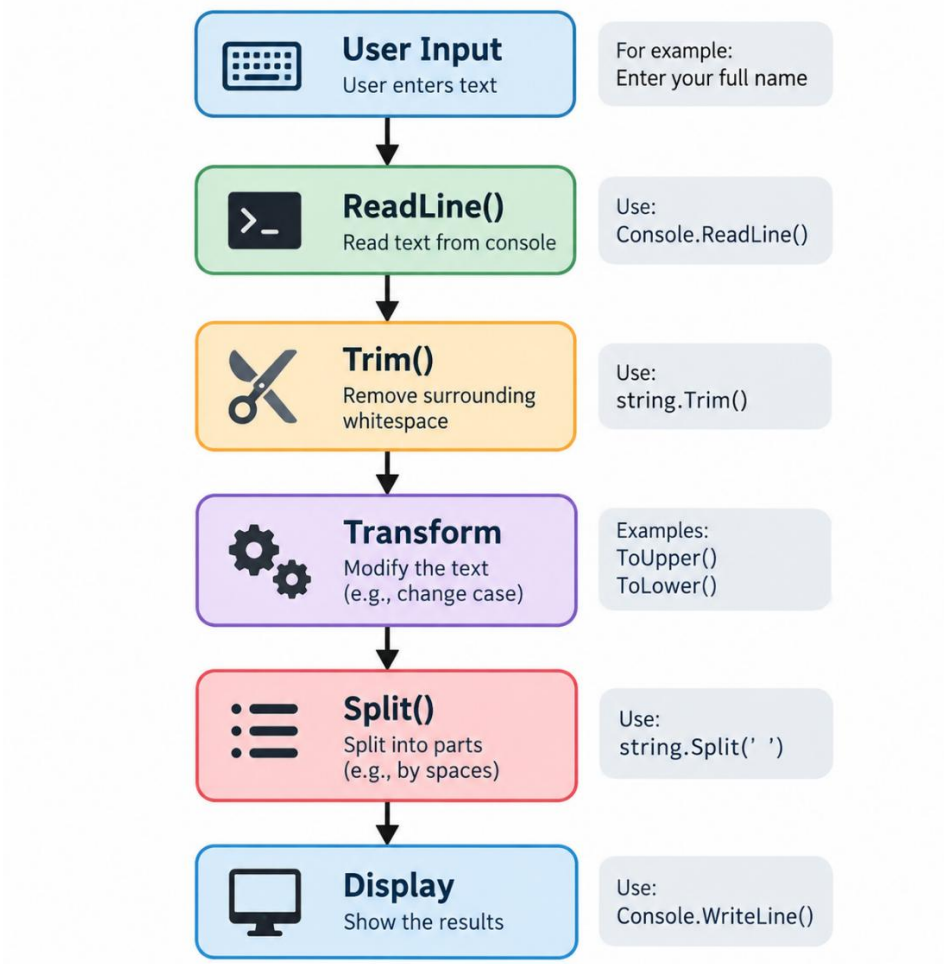
```
Console.WriteLine($"Original: {name}");
Console.WriteLine($"Uppercase: {name.ToUpper()}");
Console.WriteLine($"Lowercase: {name.ToLower()}");
```

Step 4 — Split the name

```
string[] parts = name.Split(' ');

foreach (string part in parts)
{
    Console.WriteLine(part);
}
```

This demonstrates the common data-processing sequence:



15. Why `StringBuilder` Exists

A **`StringBuilder`** provides a mutable buffer for constructing and modifying character sequences.

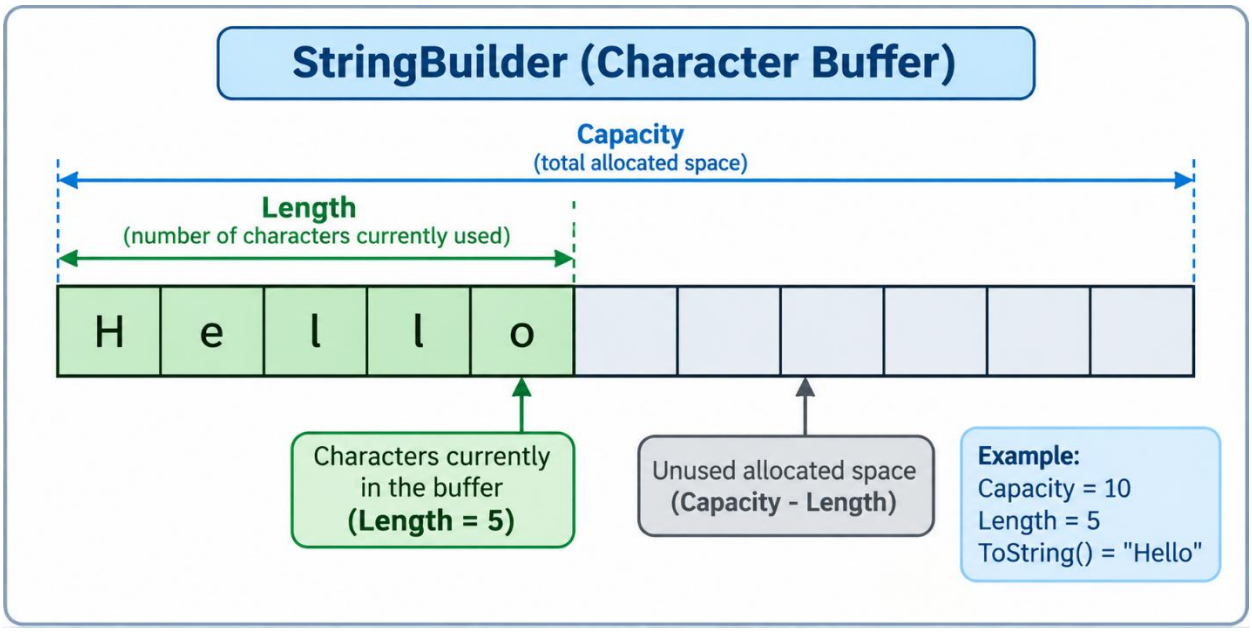
Its important members include:

- **`Capacity`** — currently allocated character space.
- **`Length`** — number of characters currently contained in the buffer.
- **`this[int index]`** — indexer providing access to a character at a given position.
- **`Append()`** — adds content to the end.
- **`Remove()`** — removes characters.
- **`Insert()`** — inserts content.
- **`Replace()`** — replaces substrings.
- **`ToString()`** — produces a string.

A capacity can also be supplied when constructing it:

```
StringBuilder builder = new StringBuilder(100);
```

This allocates an initial buffer with the specified capacity.



16. Working with `StringBuilder`

First import its namespace:

```
using System.Text;
```

Then create a builder:

```
StringBuilder sb = new StringBuilder();

sb.Append("Hello");
sb.Append(" ");
sb.Append("World");

string result = sb.ToString();

Console.WriteLine(result);
```

Output:

```
Hello World
```

Other operations include:

```
sb.Insert(0, "Message: ");
sb.Replace("World", "C#");
sb.Remove(0, 9);
```

The final `ToString()` converts the buffer into a string.

---

17. Practical Example — Extracting Capital Letters

Consider the following method:

```
public static string ExtractCapitals(string s)
{
    StringBuilder result = new StringBuilder();

    for (int i = 0; i < s.Length; i++)
    {
        if (char.IsUpper(s[i]))
        {
            result.Append(s[i]);
        }
    }

    return result.ToString();
```

```
}
```

This example demonstrates both string traversal and **`StringBuilder`**.

How It Works

The method first creates an empty buffer:

```
StringBuilder result = new StringBuilder();
```

The loop examines every character:

```
for (int i = 0; i < s.Length; i++)
```

The expression:

```
s[i]
```

retrieves the character at position **`i`**.

Then:

```
char.IsUpper(s[i])
```

checks whether that character is uppercase.

Only uppercase characters are appended:

```
result.Append(s[i]);
```

Finally:

```
return result.ToString();
```

converts the accumulated characters into a regular string.

Try It

```
Console.WriteLine(ExtractCapitals("Champlain College CSharp"));
```

You should trace the loop and determine which characters are appended before running the program.

---

18. Nullable Value Types

Ordinary value types such as **`int`** must represent a value. A reference can represent the absence of an object using **`null`**.

C# also allows a **value type to be nullable** by adding ? to the type.

```
int? age;

age = null;
The declaration:
int? age;
```
means that **age** can represent either:

```
an integer value

      OR

     null
```

This is different from using 0.

```
int? age = 0;
```

means the known age/value is zero, whereas:

```
int? age = null;
```

means no value is currently represented.

Important

0 and **null** represent different states. Do not use an arbitrary numeric value such as 0 merely to mean "unknown" when absence of a value needs to be represented explicitly.

---

19. Enumerations

An **enumeration (enum)** is a special value type that defines a set of named numeric constants.

For example:

```
public enum Region
{
    NorthAmerica,
    Europe,
    Asia
}
```
Instead of representing a region with unexplained numbers or arbitrary strings, the program can use:

```
Region.NorthAmerica
Region.Europe
Region.Asia
```

This makes the valid choices explicit in the type itself.

---

20. Using an Enum in a Class

Consider:

```
public class Department
{
    public string name;
    public Region region;
}
```

Create an object:

```
Department d = new Department();

d.name = "Computer Science";
d.region = Department.Region.NorthAmerica;
```

Because region has type Region, the program can work with the named enum values.

Testing an Enum Value

```
if (d.region == Region.NorthAmerica)
{
    Console.WriteLine("North American department");
}
```

---

21. Enum Numeric Values

Enum members have underlying numeric values.

Given:

```
public enum Region
{
    NorthAmerica,
    Europe,
    Asia
}
```

The members correspond by default to:

```
NorthAmerica → 0
Europe       → 1
Asia         → 2
```

Therefore:

```
Console.WriteLine((Region)1);
```

prints:

```
Europe
```

Note

The cast `(Region)1` converts the numeric value `1` to the corresponding `Region` value.

---

22. Flag Enumerations

A normal enum usually represents **one choice**.

For example:

```
Region region = Region.Europe;
```

Sometimes a variable must represent **multiple choices simultaneously**.

Examples include:

```
Read + Write permissions
```

```
English + French languages
```

```
Monday + Wednesday availability
```

For these situations, C# supports **flag enumerations**.

A `flag` enum should:

- use the `[Flags]` attribute;
- assign values using powers of two;
- normally include a zero-valued `None` member.

Example:

```
[Flags]
public enum Languages
{
    None = 0,
    English = 1,
    French = 2,
    Spanish = 4,
    German = 8
```

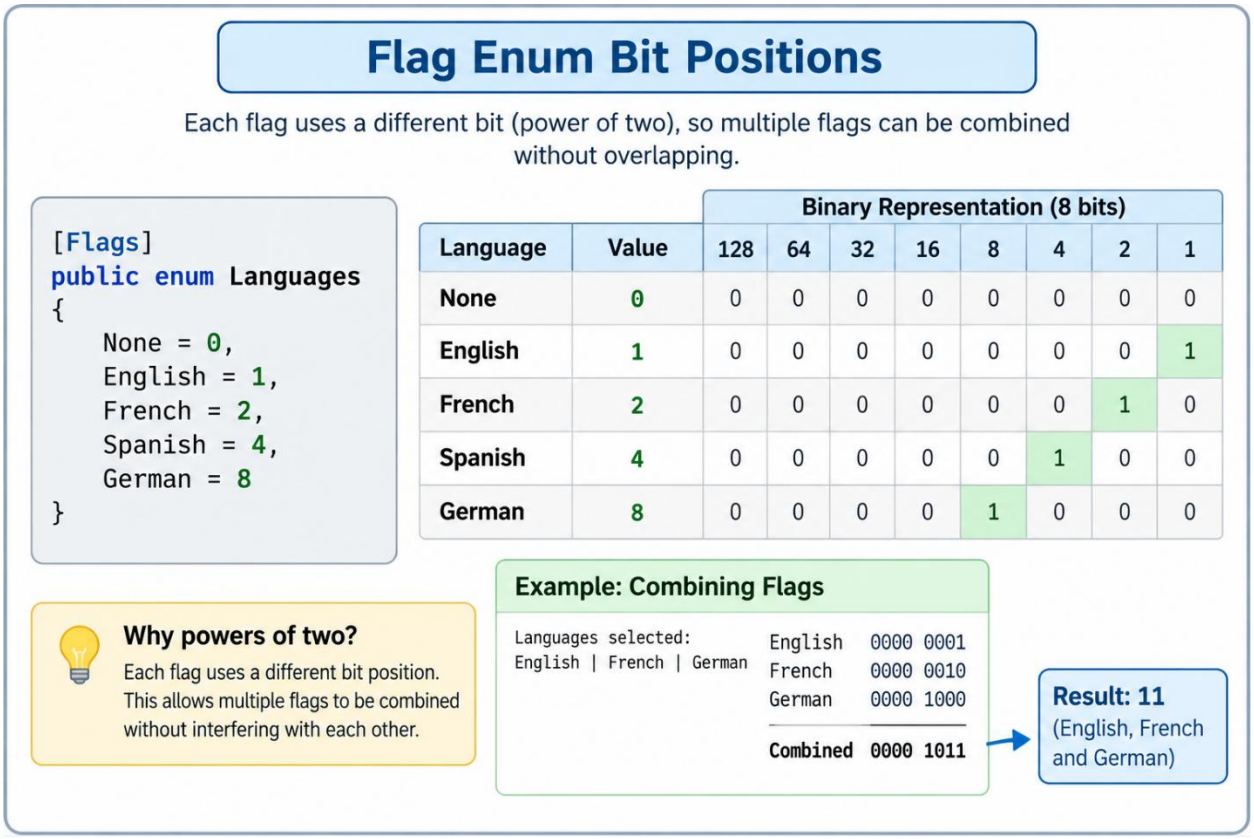
```
}
```

Why Powers of Two?

In binary:

```
English = 1  → 0001
French  = 2  → 0010
Spanish = 4  → 0100
German  = 8  → 1000
```

Each option occupies a different bit position. This allows several options to coexist without ambiguity.



---

23. Combining Flag Values with |

The bitwise OR operator | combines flags.

```
var userLanguages =
    Languages.French |
    Languages.English |
    Languages.German;
```

Conceptually:

```
English   0001
French    0010
German    1000
          ----
Combined  1011
```

A new option can be added later using `|=`:

```
userLanguages |= Languages.Spanish;
```

This adds the **`Spanish`** flag while preserving the existing selections.

---

24. Testing Flags with `&`

The bitwise AND operator **`&`** determines whether two flag sets overlap.

```
var applicantLanguage = Languages.English;

var requiredLanguage =
    Languages.French |
    Languages.English;

bool isAdmissible =
    (applicantLanguage & requiredLanguage) != 0;
```

The expression:

```
applicantLanguage & requiredLanguage
```

retains flags appearing in both values.

If the result is not zero, at least one flag overlaps.

---

25. Checking Overlap Between Flag Sets

Suppose a flag enum called **`MyDays`** defines days of the week.

```
var mondayOrTuesday =
    MyDays.Monday | MyDays.Tuesday;

var mondayOrWednesday =
    MyDays.Monday | MyDays.Wednesday;

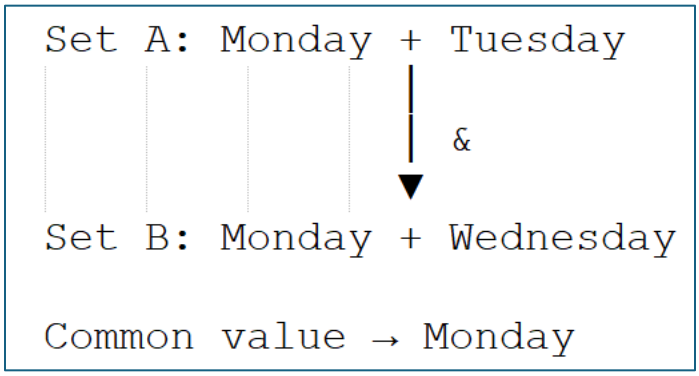
var overlap =
    mondayOrTuesday & mondayOrWednesday;
```

The common flag is:

`Monday`

Therefore, overlap represents **`MyDays.Monday`**.

Conceptually:



---

26. Practical Lab — Language Requirements

This exercise brings together flag enums and bitwise operations.

Step 1 — Define the Enum

```
[Flags]
public enum Languages
{
    None = 0,
    English = 1,
    French = 2,
    Spanish = 4,
    German = 8
}
```

Step 2 — Assign Languages to an Applicant

```
Languages applicantLanguages =

    Languages.English | Languages.French;
```

Step 3 — Define Required Languages

```
Languages requiredLanguages =

    Languages.French | Languages.German;
```

Step 4 — Check for Any Overlap

```
bool hasMatchingLanguage =
    (applicantLanguages & requiredLanguages) != 0;

Console.WriteLine($"Matching language: {hasMatchingLanguage}");
```

Step 5 — Add Another Language

```
applicantLanguages |= Languages.German;
```

Step 6 — Examine the Result

```
Console.WriteLine($"Applicant languages: {applicantLanguages}");
```

The important distinction is:

`|` combines flags

`&` finds overlapping flags

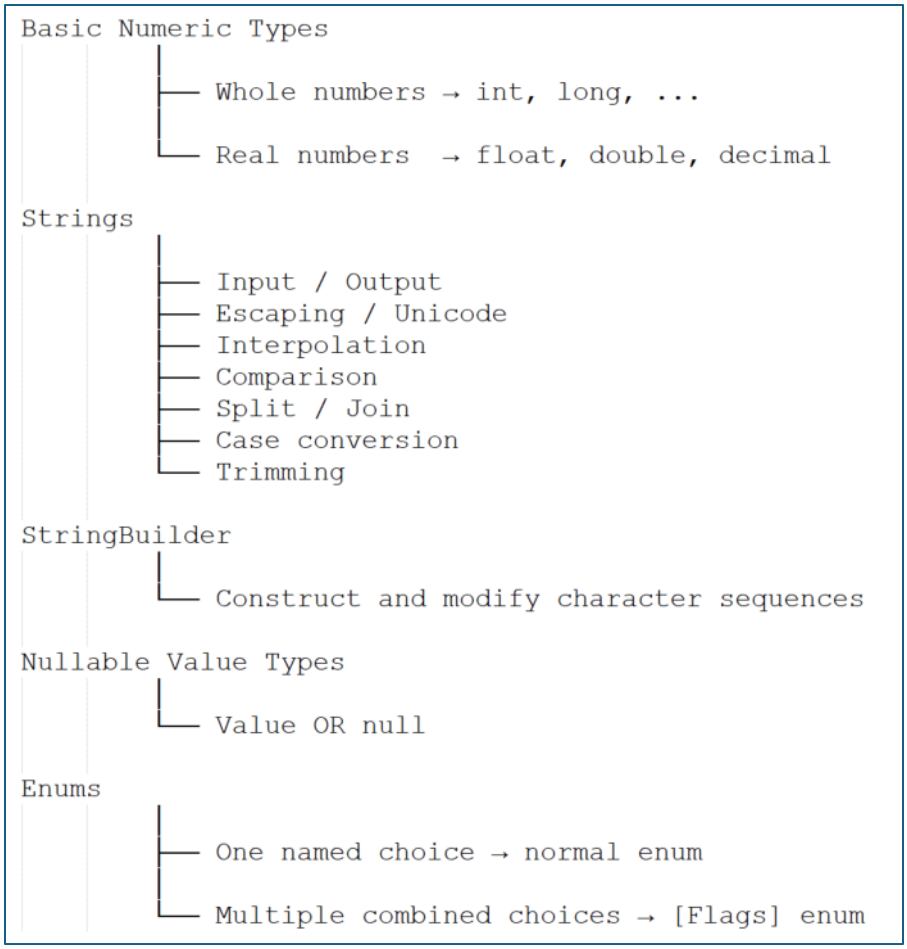
Pitfall — Flags Need Independent Bits

Values such as `1, 2, 3, 4` should not be used for independent flags because 3 already contains the bits represented by `1` and `2`. Use powers of two: `1, 2, 4, 8, 16, ....`

---

27. Putting the Concepts Together

The concepts in this section solve different data-representation problems:


Basic Numeric Types
Whole numbers → int, long, ...
Real numbers → float, double, decimal
Strings
Input / Output
Escaping / Unicode
Interpolation
Comparison
Split / Join
Case conversion
Trimming
StringBuilder
Construct and modify character sequences
Nullable Value Types
Value OR null
Enums
One named choice → normal enum
Multiple combined choices → [Flags] enum